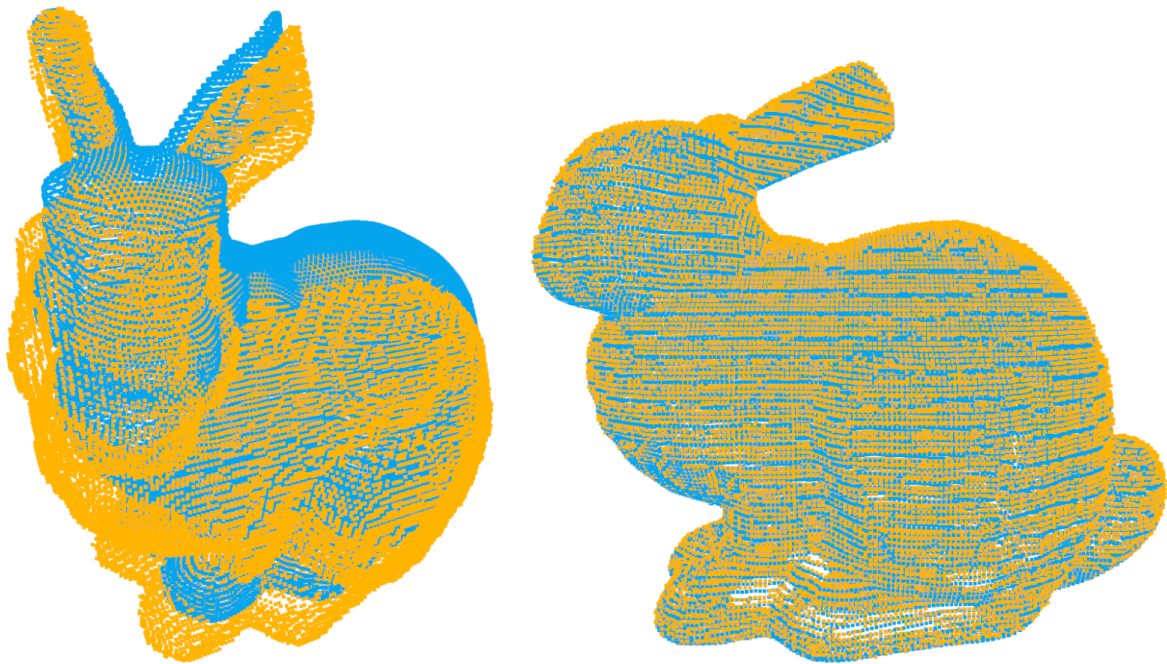


3D DATA PROCESSING - LAB 3 (*Individual assignment*)



Topic: Point Cloud Registration

Goal: Given an unaligned source and target point cloud, find the final alignment transformation from the source to the target cloud.

Instructions

You are required to extend the provided C++ software by implementing a global registration step based on feature descriptors, followed by an ICP refinement stage.

STEP1: Descriptor (Initial Alignment)

- You must detect keypoints and compute feature descriptors using Open3D functions.
- Once you have computed descriptors and found correspondences, you **may use** `RegistrationRANSACBasedOnCorrespondence` to robustly estimate the initial transformation matrix.

STEP2: ICP Refinement

- Finish to implement the ICP algorithm, including:
 - Nearest-neighbor matching (e.g., using `KDTreeFlann`)
 - Transformation estimation via SVD
 - Transformation estimation via LM
 - RMSE-based convergence checking

STEP3: ICP Methods Analysis

- The pose estimated in STEP 1 is intentionally perturbed by applying a random transformation. This perturbed pose is then used as the initial guess for the ICP

algorithm in order to simulate realistic misalignment conditions between the source and target point clouds.

- Tune ICP parameters to properly handle **small** initial misalignments.
- Compare different ICP variants, including the custom implementations developed in STEP2 (SVD-based and Levenberg–Marquardt ICP) and the methods already implemented in the code (point-to-point, point-to-plane and generalized ICP provided by Open3D).
- Evaluate and compare the ICP methods in terms of execution time, number of iterations, final RMSE and robustness to the initial misalignment.

The provided software already implements the following methods:

- `Registration(...)`
 - Initialize the source and target point cloud to be processed.
- `draw_registration_result()`
 - Visualize source and target point cloud.
- `get_transformation()`
 - Get the current transformation matrix needed to align the source to the target point cloud.
- `compute_rmse()`
 - Compute the RMSE between the points of the source and the target point.
- `execute_icp_registration(...)`
 - Main ICP loop, check convergence criteria and call `find_closest_point(...)` followed by either `get_svd_icp_registration(...)` or `get_lm_icp_registration(...)`.

Instead, the following methods must be completed in order to successfully perform ICP:

- `find_closest_point(...)`
 - For each point in the source point cloud find the closest one in the target (look at `compute_rmse()`).
- `get_svd_icp_transformation(...)`
 - First extract the centroid for each of the two point clouds, after subtracting it use `Eigen::JacobiSVD<Eigen::MatrixXd>` on the matrix obtained by multiplying the two Nx3 point matrices, ordered following the results of the `find_closest_point(...)` to successfully perform SVD decomposition.
- `get_lm_icp_transformation(...)`
 - Remember to define a templated functor that computes the distance error/residual (defined as `PointDistance`). See Ceres Solver tutorial for a

better understanding. Differently from the Bundle Adjustment only the 6-dimensional array (rx, ry, rz, tx, ty, tz) must be optimized (instead of jointly optimizing camera and 3D point positions). As in

`get_svd_icp_transformation(...)` use the point correspondences extracted previously using `find_closest_point(...)`. Remember to convert from the euler axis-angle representation to rotation matrix.

- **`execute_descriptor_registration()`**
 - Detect keypoints and compute descriptors. Then, match the descriptors and estimate a robust initial rigid transformation. Remember to store the resulting transformation in `transformation_`.
 - You may use any built-in Open3D function.

Compilation Instruction

- `mkdir build && cd build`
- `cmake ..`
- `make`

To execute:

`./registration path/to/source path/to/target mode`

where mode could be **`svd`**, **`lm`**, **`o3d-p2point`**, **`o3d-p2plane`**, **`o3d-gen`** or **`all`** (**`all`** is used to run all methods and print analysis results).

What you need to deliver

- Source code (without objects and executables).
- A .ply file for the three provided datasets, representing the registered clouds.
- A short written report with:
 - A brief description of the work done.
 - Some qualitative results (screenshots of aligned point clouds) for the two provided datasets.
 - Quantitative analysis result in term of RMSE, number of iterations and time for the three provided datasets.